

How a Java Program Runs – Behind the Scenes (Lifecycle Explained)

16 July 2025 11:00

Step 1: Writing the Source Code

```
MyClass.java

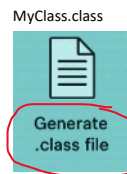
public class MyClass {
    public void display() {
        System.out.println("Hello, Java!");
    }
}
```

Step 2: Compiling with javac

```
javac MyClass.java
```

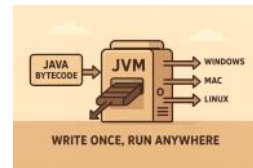
The Java compiler reads your source code line by line and checks it for syntax errors. If everything looks good, the compiler converts your source code into a special intermediate format called bytecode.

Step 3: Bytecode and Platform Independence



This bytecode is platform-independent, which means it can run on any operating system — whether it's Windows, Mac, or Linux — as long as that system has a Java Virtual Machine installed.

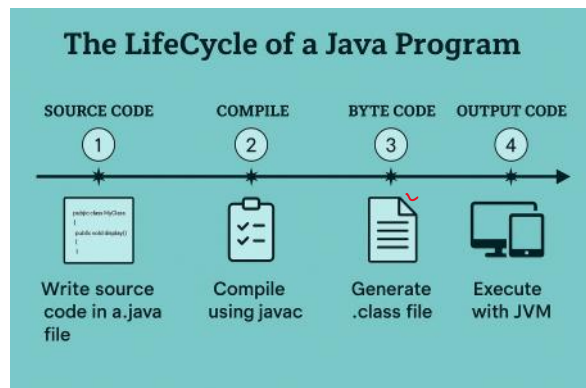
[Step 4: Execution with JVM



The JVM is the engine that runs your Java program. It acts like a translator and runtime environment that makes bytecode executable on your computer.

```
java MyClass
```

- The JVM loads the MyClass.class file.
- It performs a verification step to ensure that the bytecode is safe and valid
- Once verified, the JVM starts executing the bytecode. It can do this in two ways — by interpreting it line-by-line or by using Just-In-Time compilation, also known as JIT, to convert the bytecode into native machine code on the fly.



Write Once, Run Anywhere.

